

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Universidad Técnica Federico Santa María

Departamento de Industrias

Laboratorio 5

JOINs Avanzados y Subconsultas con MySQL

Sistemas de Información para la Gestión

Asignatura: ICN292 Sistemas de Información para la Gestión

Profesor: Patricio Cifuentes

Estudiante: Vicente Acevedo

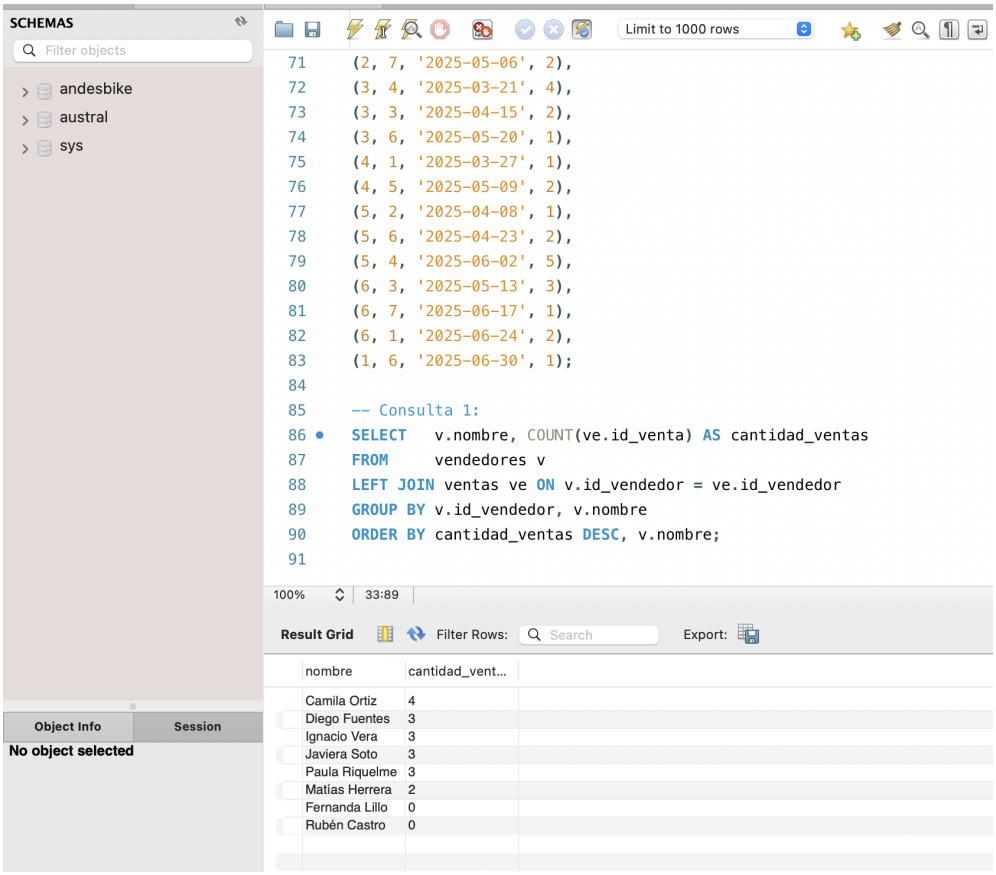
Rut: 21.430.418-K

Repositorio: `https://github.com/vacevedo3110/ICN292-Lab5-Acevedo-Vicente`

14 de noviembre de 2026

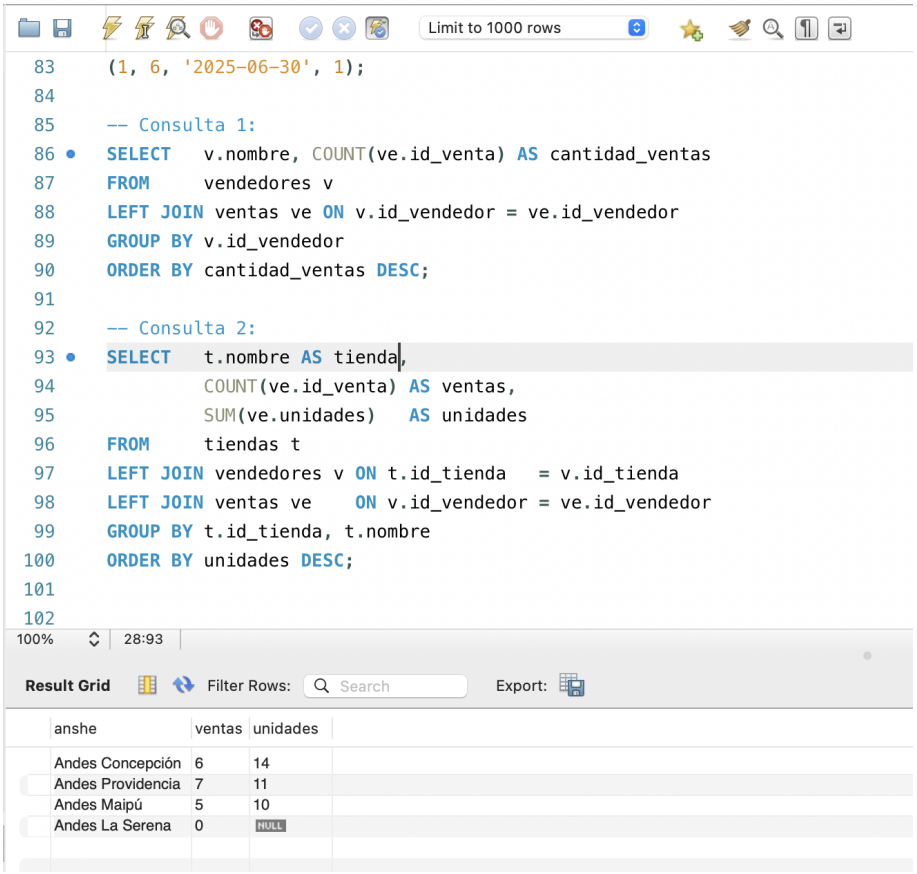
Consulta 1

```sql
SELECT   v.nombre, COUNT(ve.id_venta) AS cantidad_ventas
FROM     vendedores v
LEFT JOIN ventas ve ON v.id_vendedor = ve.id_vendedor
GROUP BY v.id_vendedor, v.nombre
ORDER BY cantidad_ventas DESC, v.nombre;
```



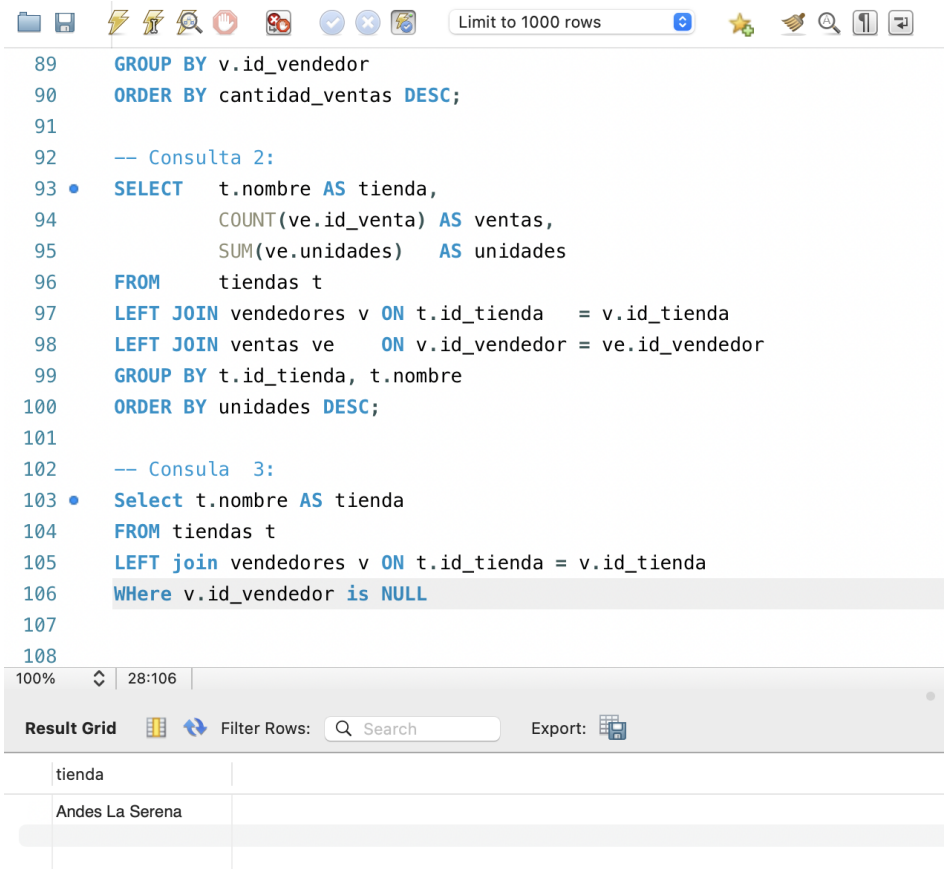
Consulta 2

```sql
SELECT   t.nombre AS tienda,
         COUNT(ve.id_venta) AS ventas,
         SUM(ve.unidades)   AS unidades
FROM     tiendas t
LEFT JOIN vendedores v ON t.id_tienda   = v.id_tienda
LEFT JOIN ventas ve    ON v.id_vendedor = ve.id_vendedor
GROUP BY t.id_tienda, t.nombre
ORDER BY unidades DESC;
```



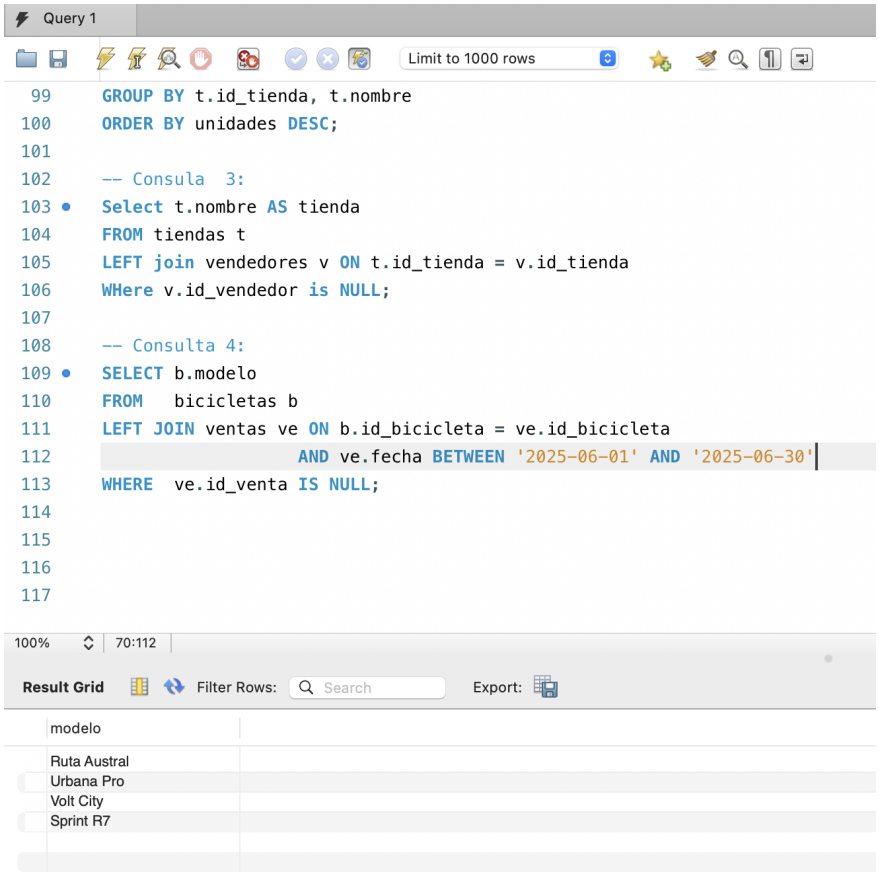
Consulta 3

```sql
SELECT t.nombre AS tienda
FROM tiendas t
LEFT JOIN vendedores v ON t.id_tienda = v.id_tienda
WHERE v.id_vendedor IS NULL;
```



Consulta 4

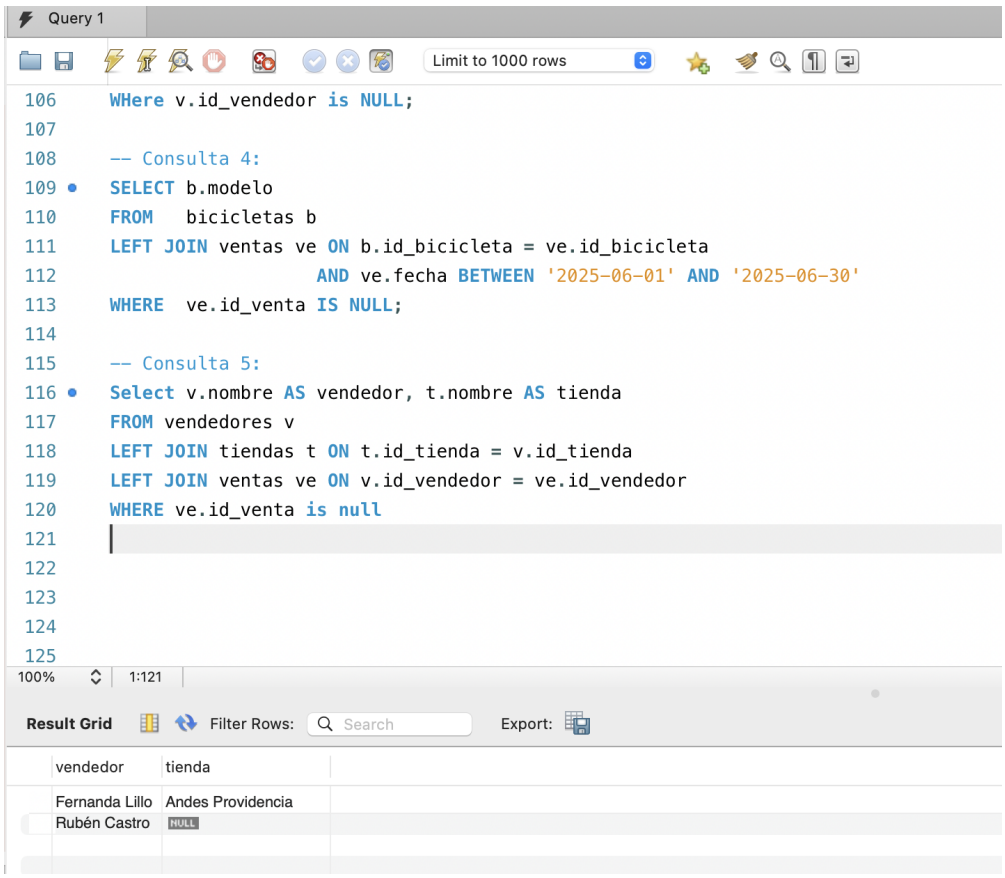
```sql
SELECT b.modelo
FROM   bicicletas b
LEFT JOIN ventas ve ON b.id_bicicleta = ve.id_bicicleta
                   AND ve.fecha BETWEEN '2025-06-01' AND '2025-06-30'
WHERE  ve.id_venta IS NULL;
```



La condición del mes se puso dentro del `ON` del `LEFT JOIN`, si se hubiera puesto en el `WHERE`, la consulta no habría devuelto ninguna fila, porque el `WHERE` elimina las filas sin venta (donde `ve.fecha` es NULL) y además contradice el filtro `ve.id_venta IS NULL`.

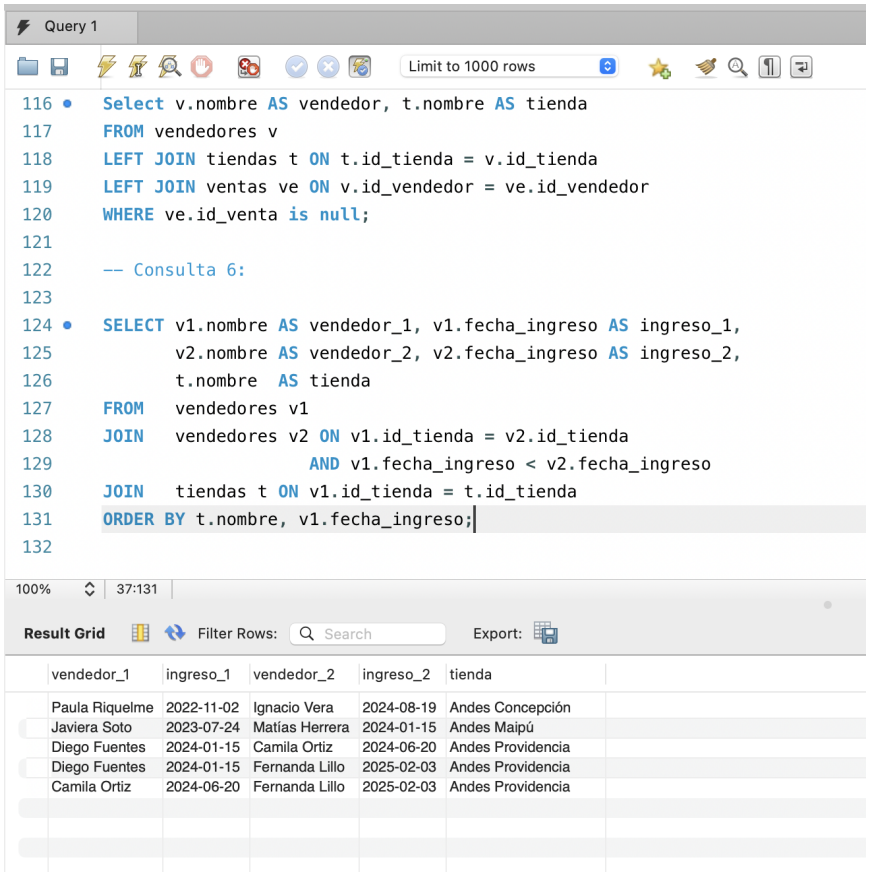
Consulta 5

```sql
SELECT v.nombre AS vendedor, t.nombre AS tienda
FROM vendedores v
LEFT JOIN tiendas t ON t.id_tienda = v.id_tienda
LEFT JOIN ventas ve ON v.id_vendedor = ve.id_vendedor
WHERE ve.id_venta IS NULL;
```



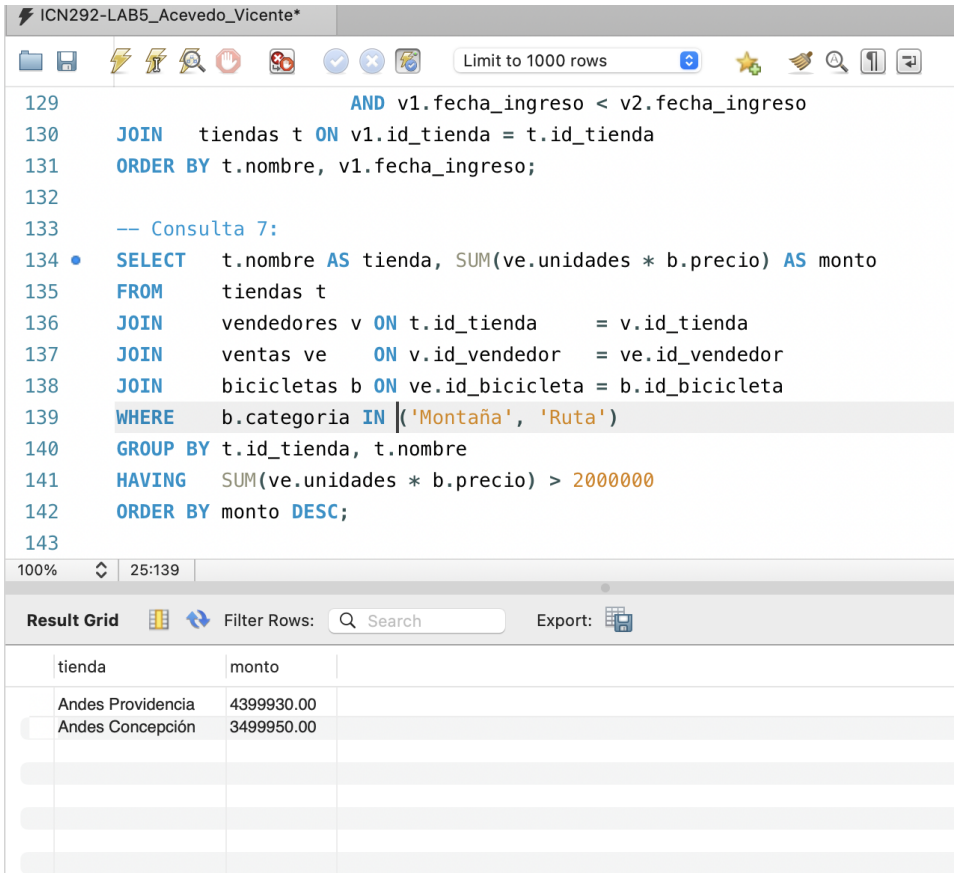
Consulta 6

```sql
SELECT v1.nombre AS vendedor_1, v1.fecha_ingreso AS ingreso_1,
       v2.nombre AS vendedor_2, v2.fecha_ingreso AS ingreso_2,
       t.nombre  AS tienda
FROM   vendedores v1
JOIN   vendedores v2 ON v1.id_tienda = v2.id_tienda
                    AND v1.fecha_ingreso < v2.fecha_ingreso
JOIN   tiendas t ON v1.id_tienda = t.id_tienda
ORDER BY t.nombre, v1.fecha_ingreso;
```



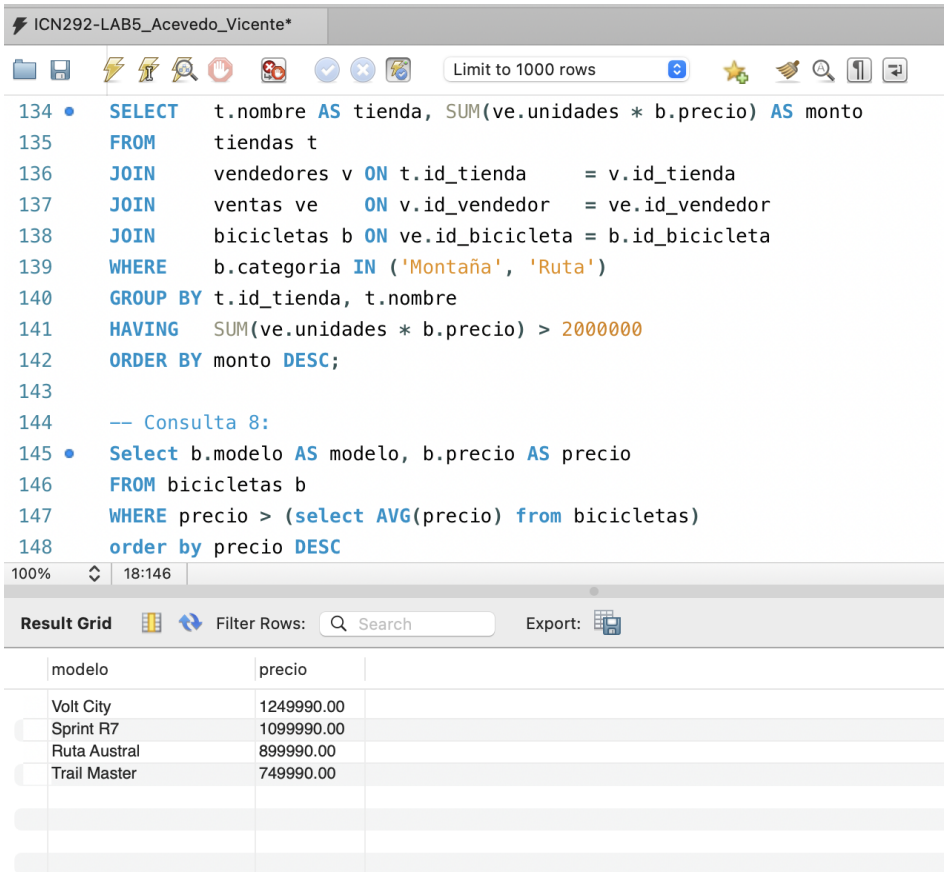
Consulta 7

```sql
SELECT   t.nombre AS tienda, SUM(ve.unidades * b.precio) AS monto
FROM     tiendas t
JOIN     vendedores v ON t.id_tienda     = v.id_tienda
JOIN     ventas ve    ON v.id_vendedor   = ve.id_vendedor
JOIN     bicicletas b ON ve.id_bicicleta = b.id_bicicleta
WHERE    b.categoria IN ('Montaña', 'Ruta')
GROUP BY t.id_tienda, t.nombre
HAVING   SUM(ve.unidades * b.precio) > 2000000
ORDER BY monto DESC;
```



Consulta 8

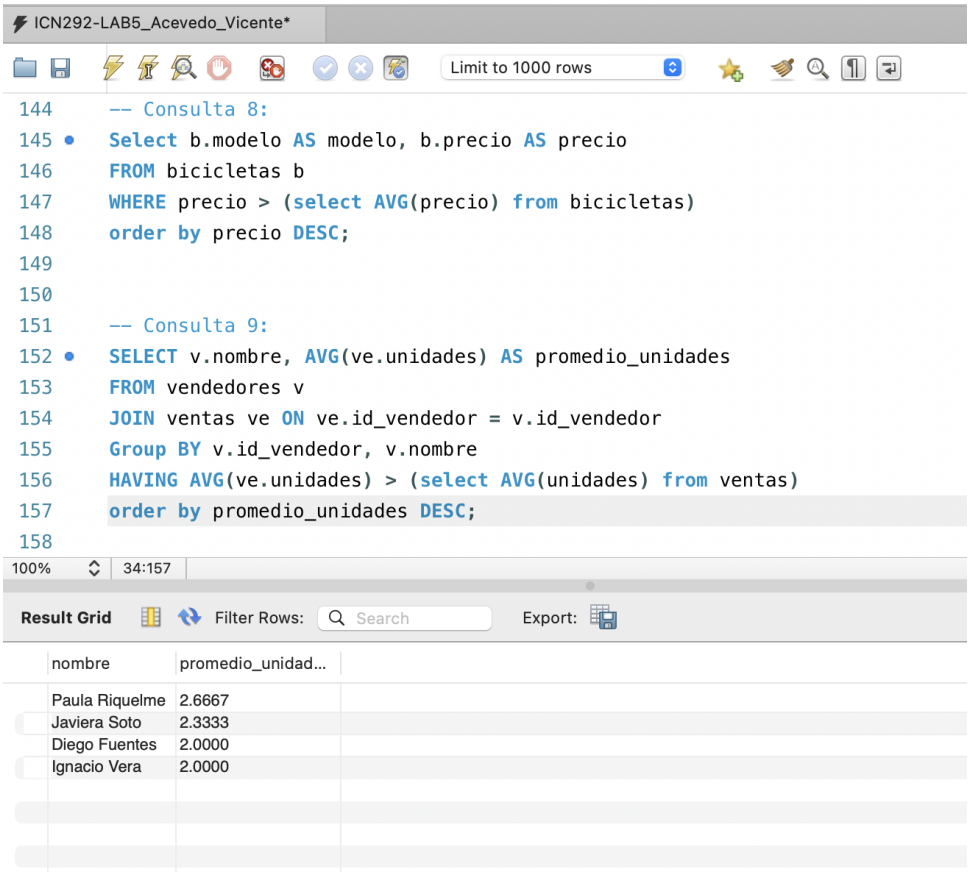
```sql
SELECT b.modelo AS modelo, b.precio AS precio
FROM bicicletas b
WHERE b.precio > (SELECT AVG(precio) FROM bicicletas)
ORDER BY b.precio DESC;
```



El promedio del catálogo es \$657.490 y se obtuvo con la subconsulta `(SELECT AVG(precio) FROM bicicletas)`, que calcula el promedio de los precios de los ocho modelos de la tabla `bicicletas`.

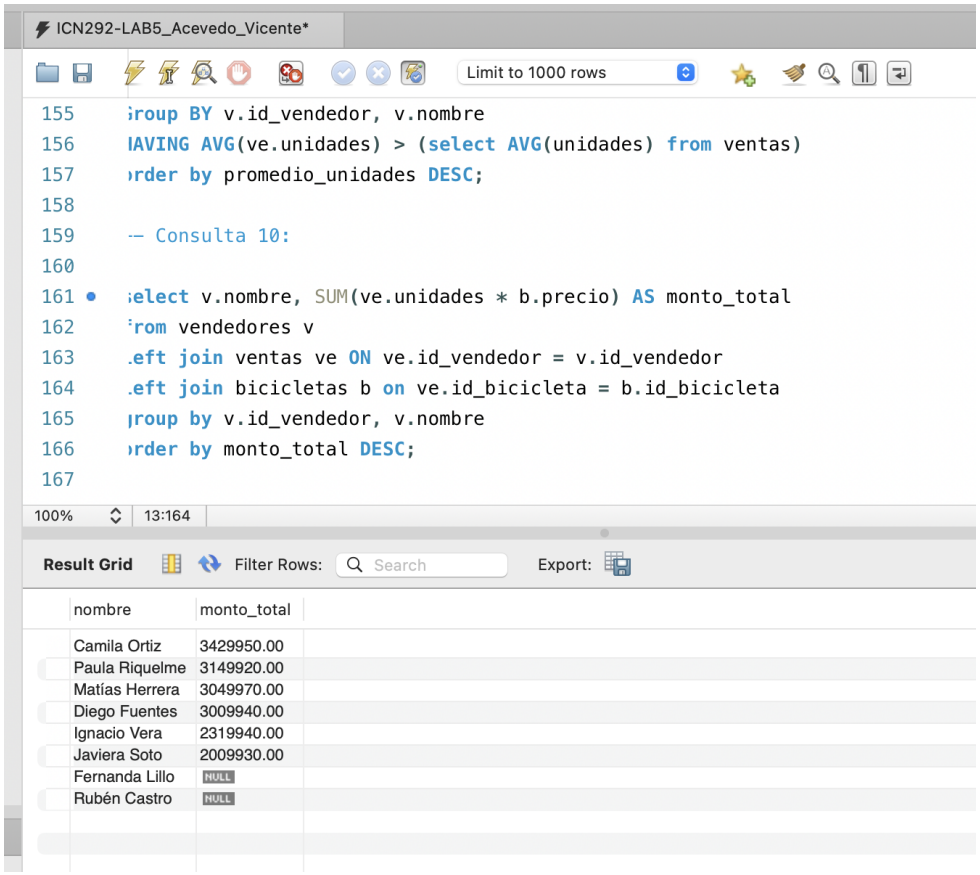
Consulta 9

```sql
SELECT v.nombre, AVG(ve.unidades) AS promedio_unidades
FROM vendedores v
JOIN ventas ve ON ve.id_vendedor = v.id_vendedor
GROUP BY v.id_vendedor, v.nombre
HAVING AVG(ve.unidades) > (SELECT AVG(unidades) FROM ventas)
ORDER BY promedio_unidades DESC, v.nombre;
```



Consulta 10

```sql
SELECT v.nombre, SUM(ve.unidades * b.precio) AS monto_total
FROM vendedores v
LEFT JOIN ventas ve ON ve.id_vendedor = v.id_vendedor
LEFT JOIN bicicletas b ON ve.id_bicicleta = b.id_bicicleta
GROUP BY v.id_vendedor, v.nombre
ORDER BY monto_total DESC;
```



Fernanda Lillo y Rubén Castro no muestran cero sino NULL porque no registraron ninguna venta, el `LEFT JOIN` los deja con valores NULL en las columnas de `ventas`, y la suma de valores NULL no es cero sino NULL (un valor desconocido). Para que mostraran cero habría que agregar a la consulta una función que reemplace el NULL por 0 en el resultado de la suma, por ejemplo `IFNULL(SUM(ve.unidades * b.precio), 0)`.